

5 Gestion des Formulaires en React

5.1 Formulaires Non Contrôlés (Approche Traditionnelle)

Commençons par voir comment fonctionnent les formulaires HTML classiques sans React.

Listing 13 – Formulaire non contrôlé (DOM directement)

```
function UncontrolledForm() {
  const handleSubmit = (event) => {
    event.preventDefault();

    // Accès direct au DOM
    const name = event.target.elements.name.value;
    const email = event.target.elements.email.value;

    console.log('Nom:', name);
    console.log('Email:', email);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" type="text" placeholder="Nom" />
      <input name="email" type="email" placeholder="Email" />
      <button type="submit">Envoyer</button>
    </form>
  );
}
```

Problèmes de cette approche :

- Pas de contrôle en temps réel
- Validation uniquement à la soumission
- Difficile de synchroniser UI avec données
- Pas de single source of truth

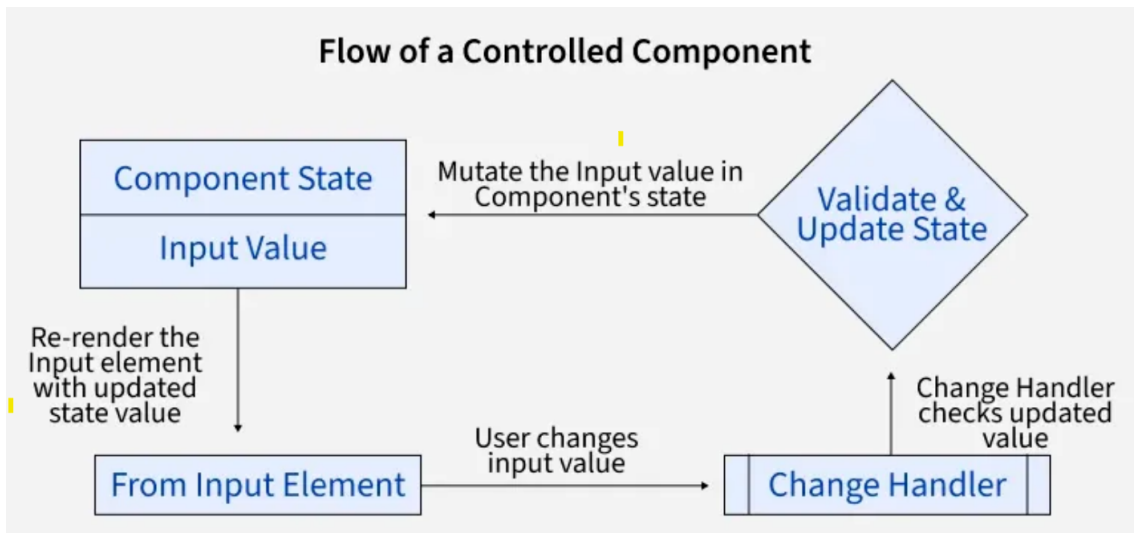
△ Important

Single Source of Truth

En React, le **state** doit être la seule source de vérité pour les données. Le DOM ne doit être qu'un reflet du state.

5.2 Formulaires Contrôlés (Approche React)

Un **formulaire contrôlé** est un formulaire dont les valeurs sont gérées par le state React.
Schéma du flux de données :



Avantages formulaires contrôlés :

- Validation en temps réel
- Contrôle total sur l'input
- Single source of truth (state)
- Facile à déboguer

5.3 Input Texte Contrôlé

Listing 14 – Input simple contrôlé

```
import { useState } from 'react';

function NameForm() {
  const [name, setName] = useState('');

  const handleChange = (event) => {
    setName(event.target.value);
  };

  return (
    <div>
      <input
        type="text"
        value={name} // Valeur liée au state
        onChange={handleChange} // Met à jour le state
        placeholder="Votre nom"
      />
      <p>Bonjour {name} !</p>
      <p>Longueur : {name.length} caractères</p>
    </div>
  );
}
```

Flux de données détaillé :

1. Utilisateur tape "A" dans l'input

2. **Event onChange** se déclenche
3. **handleChange** reçoit l'événement avec `event.target.value = "A"`
4. **setName("A")** met à jour le state
5. **Re-render** du composant
6. **Input** affiche `value={name}` soit "A"

5.4 Textarea Contrôlé

Listing 15 – Textarea avec compteur de caractères

```
function MessageForm() {
  const [message, setMessage] = useState('');
  const maxLength = 200;

  return (
    <div>
      <textarea
        value={message}
        onChange={(e) => setMessage(e.target.value)}
        placeholder="Votre message"
        rows={5}
        maxLength={maxLength}
      />
      <p>{message.length} / {maxLength} caractères</p>
    </div>
  );
}
```

★ Astuce

Différence HTML vs React :

En HTML, textarea utilise : `<textarea>Contenu</textarea>`

En React, textarea utilise value : `<textarea value={text} />`

5.5 Checkboxes

Listing 16 – Checkbox unique

```
function NewsletterForm() {
  const [acceptNewsletter, setAcceptNewsletter] = useState(false);

  return (
    <div>
      <label>
        <input
          type="checkbox"
          checked={acceptNewsletter} // checked au lieu de value
          onChange={(e) => setAcceptNewsletter(e.target.checked)}
        />
        J'accepte de recevoir la newsletter
      </label>

      <p>Newsletter : {acceptNewsletter ? 'Activée' : 'Désactivée'}</p>
    </div>
  );
}
```

Checkboxes multiples :

Listing 17 – Plusieurs checkboxes

```
function PreferencesForm() {
  const [preferences, setPreferences] = useState({
    cuisine: false,
    mode: false,
    decoration: false
  });

  const handleCheckbox = (event) => {
    const { name, checked } = event.target;
    setPreferences({
      ...preferences,
      [name]: checked
    });
  };

  return (
    <div>
      <label>
        <input
          type="checkbox"
          name="cuisine"
          checked={preferences.cuisine}
          onChange={handleCheckbox}
        />
        Cuisine
      </label>

      <label>
        <input
          type="checkbox"
          name="mode"
          checked={preferences.mode}
          onChange={handleCheckbox}
        />
        Mode
      </label>

      <label>
        <input
          type="checkbox"
          name="decoration"
          checked={preferences.decoration}
          onChange={handleCheckbox}
        />
        Décoration
      </label>

      <p>Sélections : {JSON.stringify(preferences)}</p>
    </div>
  );
}
```

5.6 Radio Buttons

Listing 18 – Radio buttons groupés

```
function GenderForm() {
  const [gender, setGender] = useState('');

  return (
    <div>
      <p>Genre :</p>

      <label>
        <input
          type="radio"
          name="gender"
          value="homme"
          checked={gender === 'homme'}
          onChange={(e) => setGender(e.target.value)}
        />
        Homme
      </label>

      <label>
        <input
          type="radio"
          name="gender"
          value="femme"
          checked={gender === 'femme'}
          onChange={(e) => setGender(e.target.value)}
        />
        Femme
      </label>

      <label>
        <input
          type="radio"
          name="gender"
          value="autre"
          checked={gender === 'autre'}
          onChange={(e) => setGender(e.target.value)}
        />
        Autre
      </label>

      {gender && <p>Sélection : {gender}</p>}
    </div>
  );
}
```

△ Important

Radio vs Checkbox :

Radio : Un seul choix parmi plusieurs → `checked={value === 'option'}`

Checkbox : Plusieurs choix possibles → `checked={boolean}`

5.7 Select (Liste déroulante)

Listing 19 – Select simple

```

function CategoryForm() {
  const [category, setCategory] = useState('');

  return (
    <div>
      <select
        value={category}
        onChange={(e) => setCategory(e.target.value)}
      >
        <option value="">-- Choisir une catégorie --</option>
        <option value="cuisine">Cuisine</option>
        <option value="mode">Mode</option>
        <option value="decoration">Décoration</option>
        <option value="artisanat">Artisanat</option>
      </select>

      {category && <p>Catégorie : {category}</p>}
    </div>
  );
}

```

Select multiple :

Listing 20 – Select avec sélection multiple

```

function MultiSelectForm() {
  const [selectedCategories, setSelectedCategories] = useState([]);

  const handleSelectChange = (event) => {
    const options = event.target.options;
    const selected = [];

    for (let i = 0; i < options.length; i++) {
      if (options[i].selected) {
        selected.push(options[i].value);
      }
    }

    setSelectedCategories(selected);
  };

  return (
    <div>
      <select
        multiple
        value={selectedCategories}
        onChange={handleSelectChange}
      >
        <option value="cuisine">Cuisine</option>
        <option value="mode">Mode</option>
        <option value="decoration">Décoration</option>
      </select>

      <p>Sélections : {selectedCategories.join(', ')}</p>
    </div>
  );
}

```